

✓ Bloco A — Processamento de dados

Questão 1. Estratégia para um dataset grande

Você recebeu um conjunto de arquivos CSV com aproximadamente 800 GB no total, distribuídos em ~12 mil arquivos, com colunas inconsistentes entre eles (algumas faltam, outras têm o mesmo dado com nome diferente). Precisa transformá-los em uma base consultável e fazer agregações diárias por cliente.

Que ferramentas/abordagens você usaria e por quê? Compare ao menos duas opções concretas (ex.: Spark vs. DuckDB vs. processamento por lotes em Python).

Apache Spark é utilizado em contextos de Big Data adicionando uma complexidade maior nas configurações do sistema para um servidor único de 800GB. Python em lotes para processar cada arquivo com Pandas é bom, mas dá mais trabalho e mais código para escrever, faria sentido se o schema for irregular e precisar de tratamento customizado em cada arquivo. DuckDB seria uma boa escolha para esse caso pois lê CSV e possui funções e parâmetros para resolver schemas inconsistentes.

Como lidaria com a inconsistência do schema? Descreva o processo, não só o nome de uma técnica.

Etapa 1 — Mapeamento: scan para descobrir todas as variações

```
import duckdb
# Descubra todas as colunas únicas nos arquivos
duckdb.sql("""
    SELECT column_name, count(*) as arquivos
    FROM (
        SELECT unnest(columns) as column_name
        FROM duckdb_columns()
        WHERE table_name LIKE 'sample%'
    )
    GROUP BY column_name
    ORDER BY arquivos DESC
""")
```

Etapa 2 — Mapeamento explícito de colunas com mesmo dado mas nomes diferentes:

```
# 'valor', 'value', 'amount' são a mesma coisa
SELECT
    COALESCE(valor, value, amount) as valor_transacao,
    COALESCE(cliente_id, customer_id, client_id) as cliente_id,
    ...
FROM read_csv('*.csv', union_by_name=true)
```

Etapa 3 — Validação: contar nulos por coluna após a união para detectar colunas que não foram mapeadas corretamente.

Onde armazenaria o resultado intermediário e o final? Justifique formato e local.

Eu salvaria os arquivos em parquet porque é um formato de arquivo colunar. Em CSV o armazenamento é feito por linhas, todos os campos de uma linha ficam juntos e para ler o valor de uma coluna é obrigatório ler o registro inteiro. Em Parquet o armazenamento é feito em colunas, e com isso é possível ler apenas o bloco da coluna. Se eu quiser calcular a média de valor por dia eu não precisarei ler os 800GBs e sim somente os blocos de valor e data. Parquet pode ser consultado por DuckDB direto no arquivo.

Parquet particionado por data e cliente_id:

```
/output/
data=2026-05-01/
  cliente_id=001/
    part-0001.parquet
  cliente_id=002/
    part-0001.parquet
```

Questão 2. SQL — janela e otimização

Dada uma tabela transações(id, cliente_id, valor, data_hora) com 2 bilhões de linhas, escreva uma consulta SQL que retorne, para cada cliente, o ticket médio dos últimos 30 dias e a variação percentual em relação aos 30 dias anteriores.

Apresente a query.

```
WITH periodos AS (
  SELECT
    cliente_id,
    AVG(CASE
      WHEN data_hora >= NOW() - INTERVAL '30 days'
      THEN valor
    END) AS ticket_atual,

    AVG(CASE
      WHEN data_hora >= NOW() - INTERVAL '60 days'
      AND data_hora < NOW() - INTERVAL '30 days'
      THEN valor
    END) AS ticket_anterior

  FROM transacoes
  WHERE data_hora >= NOW() - INTERVAL '60 days'
  GROUP BY cliente_id
)
SELECT
  cliente_id,
  ROUND(ticket_atual, 2) AS ticket_medio_30d,
  ROUND(ticket_anterior, 2) AS ticket_medio_30d_anterior,
  ROUND(
    (ticket_atual - ticket_anterior)
    / NULLIF(ticket_anterior, 0) * 100
  , 2) AS variacao_percentual
FROM periodos
WHERE ticket_atual IS NOT NULL;
```

Quais índices ou particionamentos você sugeriria? Justifique.

Índice composto — índice em duas ou mais colunas:

o índice composto nessa ordem permite que o banco localize rapidamente as transações recentes de cada cliente sem varrer a tabela inteira.

```
CREATE INDEX idx_transacoes_cliente_data
ON transacoes (cliente_id, data_hora DESC);
```

Particionamento por data:

Particionar por mês faz o Postgres ler só as 2 partições dos últimos 60 dias em vez da tabela inteira

```
CREATE TABLE transacoes (
  id BIGINT,
  cliente_id INT,
  valor DECIMAL,
  data_hora TIMESTAMP
) PARTITION BY RANGE (data_hora);

CREATE TABLE transacoes_2026_05
PARTITION OF transacoes
FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');
```

O que mudaria se essa tabela estivesse em um data lake (Parquet em S3/MinIO) em vez de Postgres?

Em um data lake não existem índices, a forma de otimização muda:

Partition pruning por diretório: os arquivos ficam organizados em transacoes/ano=2026/mes=05/. DuckDB ou Spark leem só os diretórios relevantes, ignorando o resto.

Predicate pushdown: o Parquet armazena estatísticas de min/max por bloco de dados. Se você filtra data_hora >= 2026-05-01, o engine pula blocos cujo max é menor que essa data sem ler o bloco inteiro.

Projeção colunar: como o Parquet é colunar, a query lê só as colunas cliente_id, valor e data_hora e ignora todas as outras.

Questão 3. Pandas vs. Spark vs. Polars vs. DuckDB

Descreva quando você escolheria cada uma dessas ferramentas. Para cada uma, dê um caso de uso concreto onde ela é a melhor escolha e um onde ela é a pior. O critério aqui não é favoritismo — é trade-off.

Pandas: Bom para dataset pequeno e possui integrações com ferramentas do ecossistema de ciência de dados, usado para fazer previsões, explorar relatórios e fazer visualizações dos dados. Não é bom para manipular arquivos muito grandes pois o processo pode dar problemas de memória, MemoryError.

Polars: Bom para processamento em uma máquina só com grande volume de dados. Um ponto negativo é que o DataFrame não é compatível com algumas bibliotecas do Pandas para ciência de dados como scikit-learn, matplotlib e etc.

DuckDB: Bom para rodar SQL em arquivos Parquet ou CSV direto no terminal. Para pipelines que exigem programação com Python é melhor usar Polars.

Spark: Spark é bom para volumes de TBs de dados, grande processamento e cluster com múltiplas máquinas. Exige uma expertise maior dos profissionais na ferramenta. Geralmente não é usado para servidor único com volume moderado de dados pois adiciona um aumento de complexidade que pode não ser necessário.

Questão 4. Idempotência em pipelines

Explique o que é um job idempotente e por que isso importa em um pipeline diário. Dê um exemplo de problema causado pela falta de idempotência e descreva duas estratégias diferentes para resolver (não apenas "deletar antes de inserir").

Um job é idempotente quando pode ser executado mais de uma vez e sempre irá produzir o mesmo resultado. Um exemplo de um job não idempotente seria quando, o servidor cai no meio da noite e o job rodou parcialmente, ao ser executado pela manhã surgem registros duplicados na tabela, alguns clientes aparecem duas vezes, o dashboard mostra valores de receita dobrados e entre outros. Isso pode ser um grande problema pois um executivo pode tomar decisões de negócio erradas pois os dados estão incorretos nos relatórios. Um job idempotente é importante pois imprevistos e problemas tecnológicos podem acontecer e caso seja necessário reprocessar os dados é preciso ter a confiança de que eles vão sair corretamente.

Estratégia 1 — Escrita atômica via arquivo temporário:

replace() é uma operação indivisível no sistema de arquivos Linux, não existe momento em que o arquivo está ausente ou corrompido

```
tmp_path = output_path.with_suffix(".tmp.parquet")
df.write_parquet(tmp_path)      # escreve no temporário
tmp_path.replace(output_path)   # substitui atômicamente
```

Estratégia 2 — Delete por partição antes do Insert:

```
# Deleta SÓ o período que será reprocessado
DELETE FROM metricas_diarias
WHERE data = '2026-05-01';

#Insere os dados corretos
INSERT INTO metricas_diarias
SELECT cliente_id, SUM(valor) as total
FROM transacoes
WHERE data = '2026-05-01'
GROUP BY cliente_id;
```

Estratégia 3 — UPSERT / MERGE:

Se o registro já existe, atualiza. Se não existe, insere.

```
INSERT INTO metricas_diarias (data, cliente_id, total)
VALUES ('2026-05-01', 001, 500)
ON CONFLICT (data, cliente_id)
DO UPDATE SET total = EXCLUDED.total;
```

Questão 5. Diagnóstico de servidor lento

Um servidor Ubuntu 22.04 que roda um job de ETL noturno começou a estourar o horário, ficando lento sem alterações recentes no código. Descreva, em ordem, os comandos que você executaria para diagnosticar. Para cada comando, diga o que está procurando e o que esperaria ver em um sistema saudável.

```
# geral
uptime
O que procuro: load average menor que o número de CPUs (verifico com nproc)
Sistema saudável: load average 1.2, 1.1, 1.0 com 4 CPUs = OK

# CPU
top / htop
O que procuro: processo consumindo 90%+ de CPU continuamente
Sistema saudável: job usa CPU durante processamento e libera depois

# Memória
free -h
O que procuro: swap sendo usado ativamente = RAM acabou
Sistema saudável: available > 20% do total, swap próximo de zero

# I/O de disco
iostat -x 1 5
O que procuro: %util próximo de 100%, await alto
Sistema saudável: %util abaixo de 70%

df -h
O que procuro: disco cheio – causa falhas silenciosas
Sistema saudável: espaço livre acima de 10%

# Rede
ss -s
O que procuro: muitas conexões em CLOSE_WAIT ou TIME_WAIT
Sistema saudável: maioria em ESTABLISHED

# Logs do sistema
journalctl -xe / dmesg
O que procuro: "Out of memory: Kill process", "I/O error"
Sistema saudável: sem erros críticos recentes

# Logs do ETL
journalctl -u etl.service --since "today"
O que procuro: onde o job travou, qual etapa está demorando

psql -c "SELECT 1"          banco responde rápido?
ping banco-host             latência de rede ok?
curl -w "%{time_total}" API externa lenta?
pg_stat_activity            queries travadas em lock wait?
```

Questão 6. Permissões e segurança

Qual a diferença prática entre executar um serviço como root, com usuário dedicado de sistema, e dentro de um container? Quando cada um é apropriado?

executar com root: tem acesso total ao sistema, pode ler, escrever e apagar qualquer arquivo, instalar pacotes, modificar configurações e etc. Caso exista alguma vulnerabilidade no sistema e um atacante conseguir ter este acesso privilegiado ele terá muitas permissões para usufruir.

usuário dedicado: tem acesso somente aos privilégios mínimos necessários para aquela aplicação. Caso o sistema fique comprometido o atacante só terá acesso ao que está liberado para este usuário executar e não a todas as permissões de um usuário root.

Container:

Adiciona uma camada de isolamento além do usuário dedicado. O processo fica em um namespace separado, com filesystem próprio, sem acesso ao host por padrão. Mesmo que o processo dentro do container seja root, ele não é root no host. Combinando usuário dedicado + container temos a abordagem mais segura.

Cite ao menos quatro configurações de hardening que você aplicaria em um servidor Ubuntu exposto à internet (SSH, firewall, atualizações, fail2ban, etc.) e explique o porquê de cada uma.

```
# /etc/ssh/sshd_config
PermitRootLogin no
PasswordAuthentication no
```

Firewall com UFW:

```
ufw default deny incoming
ufw allow 22/tcp
ufw enable
```

Atualizações automáticas de segurança:

```
apt install unattended-upgrades
dpkg-reconfigure unattended-upgrades
```

O que faz o comando `chmod 4755` binário? Por que ele pode ser perigoso? Dê um exemplo legítimo de uso.

`chmod 4755:`

4 — setuid bit: executa com privilégios do dono do arquivo

7 — dono tem rwx

5 — grupo tem r-x

5 — outros têm r-x

O setuid faz o binário executar com os privilégios do dono. Se o dono for root, qualquer usuário executa esse binário como root. Se o binário tiver vulnerabilidades, qualquer usuário do sistema pode explorar para ganhar acesso root.

Exemplo legítimo: o comando `passwd`. Qualquer usuário precisa modificar [/etc/shadow](#) para trocar sua própria senha, mas [/etc/shadow](#) só pode ser escrito por root. O setuid no `passwd` permite essa operação específica com privilégio de root, sem dar acesso root geral.

Questão 7. Systemd

Escreva um arquivo de unit do systemd para rodar um script Python [/opt/etl/run.py](#) como usuário `etl`, com restart automático em caso de falha (com backoff), log direcionado ao journal, limite de memória de 4 GB e limite de CPU. Explique cada diretiva e diga o que aconteceria se você omitisse cada uma.

diretiva

```
[Unit]
# Nome legível nos logs. Sem isso aparece vazio no journalctl
Description=ETL diário de logs de acesso

# Inicia depois do Docker. Sem isso docker run falha se Docker não estiver pronto
After=docker.service

# Se Docker parar, esse serviço para junto. Sem isso o job tenta rodar sem Docker e falha silenciosamente
Requires=docker.service

[Service]
# Job que roda e termina. Sem isso o systemd esperaria o processo ficar em background indefinidamente
Type=oneshot

# Roda como usuário etl. Sem isso roda como root — risco de segurança
User=etl
Group=etl

# Comando a executar. Sem isso o service não faz nada
ExecStart=/opt/etl/run_daily.sh

# Logs capturados pelo journald. Sem isso os logs somem e não aparecem no journalctl
StandardOutput=journal
StandardError=journal

# Mata o processo se passar de 4GB RAM. Sem isso pode consumir toda a RAM e travar o servidor
MemoryMax=4G
```

```
# Limita CPU a 80%. Sem isso o job pode monopolizar o processador e afetar outros serviços
CPUQuota=80%

# Reinicia se sair com erro. Sem isso a falha é silenciosa – ninguém percebe até o dia seguinte
Restart=on-failure

# Espera 1 minuto antes de tentar de novo. Sem isso entra em loop de restart rápido
RestartSec=60s

[Install]
# Ativa no boot em modo normal. Sem isso o service não inicia automaticamente no boot
WantedBy=multi-user.target
```

timer

```
[Unit]
Description=Timer para ETL diário às 03:00

[Timer]
# Roda todo dia às 03:00. Formato: ano-mês-dia hora:min:seg. * significa qualquer valor
OnCalendar=*-*-* 03:00:00

# Se o servidor estava desligado às 03:00, executa assim que ligar. Sem isso o dia é pulado
Persistent=true

[Install]
# Ativa o timer no boot. Sem isso o timer não inicia automaticamente
WantedBy=timers.target
```

Questão 8. Docker e Kubernetes

Quando faz sentido containerizar um job de processamento de dados? E quando não faz?

Faz sentido: O job tem dependências específicas de versão que podem conflitar com outros serviços do servidor. Garante que o ambiente de desenvolvimento seja idêntico ao de produção "funciona na minha máquina" deixa de existir. O job precisa rodar em diferentes servidores sem reconfiguração, a imagem Docker é portátil. Garante isolamento de segurança, o container não acessa o host por padrão.

Não faz sentido: O job é um script simples sem dependências externas. O servidor tem recursos muito limitados. O job precisa de acesso direto a hardware, GPU, dispositivos físicos, interfaces de rede especiais.

Diferencie um Deployment, um Job e um CronJob no Kubernetes. Qual você usaria para um ETL diário e por quê? **bold text**

Deployment: Mantém um número fixo de pods rodando continuamente. Se um pod morrer, o Kubernetes sobe outro automaticamente. Usado para serviços que precisam estar sempre disponíveis APIs, servidores web, workers de fila.

Exemplo: replicas: 3 de uma API REST. Se um pod cair, os outros dois continuam atendendo e o K8s sobe um terceiro.

Job: Roda um ou mais pods até a tarefa completar com sucesso e termina. Garante que a tarefa foi executada pelo menos uma vez. Usado para tarefas pontuais, migração de banco, processamento único, envio de email em massa.

CronJob: Cria Jobs automaticamente em um schedule definido. É o equivalente do cron do Linux dentro do Kubernetes.

Para um ETL diário, usaria CronJob: agenda o job para rodar todo dia às 03:00, cria um Job que executa o container, e termina.

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: etl-diario
spec:
  schedule: "0 3 * * *" # todo dia às 03:00
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: etl
              image: etl-logs:latest
              args: ["--start", "$(date +%Y-%m-%d)", "--end", "$(date +%Y-%m-%d)"]
              restartPolicy: OnFailure
```

O que é um volume mount e por que ele importa em workloads de dados? Diferencie volumes efêmeros e persistentes.

Volume mount é um diretório montado dentro do container que persiste além do ciclo de vida do container. Sem volumes, tudo que o container escreve some quando ele termina. O Parquet gerado pelo container fica no host mesmo depois do container terminar.

```
-v "$(pwd)/data:/app/data"
```

Volumes efêmeros (emptyDir no K8s): Existem só enquanto o pod está rodando. Se o pod morrer, os dados somem. Úteis para dados temporários compartilhados entre containers do mesmo pod como cache, arquivos intermediários.

Volumes persistentes (PersistentVolume no K8s / bind mount no Docker): Existem independente do ciclo de vida do container. Para workloads de dados são essenciais, pois o output do ETL precisa sobreviver ao container.

Cite um cenário em que rodar o job em uma VM Ubuntu pura é melhor do que em Kubernetes.

Quando o job é simples, roda em um servidor único e o time não tem expertise em Kubernetes. K8s faz sentido quando você tem múltiplos serviços, múltiplos servidores, necessidade de escalabilidade automática.

Questão 9. Backup e recuperação

Descreva uma estratégia de backup para um banco Postgres em produção que recebe escrita constante. Aborde frequência, retenção, validação dos backups e teste de restore. Em quanto tempo, em horas, sua estratégia conseguiria restaurar o serviço após perda total da VM? E em caso de corrupção lógica detectada 3 dias depois?

Backup base diário com pg_basebackup

```
pg_basebackup \
-D /backup/base/$(date +%Y%m%d) \
-Ft -z -P \
--wal-method=stream
```

WAL archiving contínuo

```
# postgresql.conf
archive_mode = on
archive_command = 'cp %p /backup/wal/%f'
wal_level = replica
```

Retenção:

- Backups base: 30 dias
- Arquivos WAL: 7 dias
- Backups semanais: 3 meses
- Backups mensais: 1 ano

Validação dos backups: Compara o resultado com a produção. Se divergir, o backup está corrompido.

```
# Restaura o backup base
pg_restore -d postgres_test /backup/base/latest

# Verifica integridade
psql -d postgres_test -c "SELECT COUNT(*) FROM tabela_principal"
psql -d postgres_test -c "SELECT MAX(created_at) FROM tabela_principal"
```

Perda total da VM

Restaura o backup base do dia anterior + replay dos WALs até o momento da falha. Estimativa de 2 a 4 horas dependendo do tamanho do banco e velocidade do storage.

```
# Restore do backup base
pg_basebackup --pgdata=/var/lib/postgresql/data --recover-target-time="2026-05-24 03:00:00"

# postgresql.conf para PITR
```

```
restore_command = 'cp /backup/wal/%f %p'
recovery_target_time = '2026-05-24 16:30:00'
```

Corrupção lógica detectada 3 dias depois

Restaura o backup base de 4 dias atrás e faz replay dos WALs até 1 minuto antes da corrupção acontecer Point-in-Time Recovery (PITR). Estimativa de 4 a 8 horas dependendo do volume de WAL.

✓ Bloco C — Pipelines e orquestração

Questão 10. Airflow ou alternativa

Você precisa orquestrar um pipeline que tem: ingestão de uma API a cada 15 minutos, um job pesado em Spark diário, e um envio de relatório por e-mail no fim do dia. Como você modelaria isso no Airflow (ou em outra ferramenta de sua preferência)? Cite DAGs, dependências, tratamento de falha, retries e o que acontece se o job diário falhar — o relatório sai ou não?

DAG 1 — ingestão frequente

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta

dag_ingestao = DAG(
    'ingestao_api',
    schedule_interval='*/15 * * * *', # a cada 15 minutos
    start_date=datetime(2026, 1, 1),
    catchup=False, # não tenta recuperar runs perdidos
    default_args={
        'retries': 3,
        'retry_delay': timedelta(minutes=2),
    }
)

task_ingestao = PythonOperator(
    task_id='busca_api',
    python_callable=buscar_api,
    dag=dag_ingestao,
)
```

DAG 2 — processamento diário:

```
from airflow.providers.apache.spark.operators.spark_submit import SparkSubmitOperator
from airflow.operators.email import EmailOperator

dag_diario = DAG(
    'etl_diario',
    schedule_interval='0 2 * * *', # todo dia às 02:00
    start_date=datetime(2026, 1, 1),
    catchup=False,
    default_args={
        'retries': 2,
        'retry_delay': timedelta(minutes=10),
        'email_on_failure': True,
        'email': ['time@empresa.com'],
    }
)

task_spark = SparkSubmitOperator(
    task_id='job_spark',
    application='/opt/spark/etl.py',
    dag=dag_diario,
)

task_relatorio = EmailOperator(
    task_id='envia_relatorio',
    to='time@empresa.com',
    subject='Relatório diário {{ ds }}',
    html_content='Relatório do dia {{ ds }} em anexo.',
    trigger_rule='all_success', # só executa se task_spark teve sucesso
    dag=dag_diario,
)
```



```
# Define a dependência
task_spark >> task_relatorio
```

Caso o job diário falhar o relatório não sai. A linha `task_spark >> task_relatorio` define que `task_relatorio` depende de `task_spark`. Se `task_spark` falhar após todos os retries, `task_relatorio` fica em estado `upstream_failed` e não executa.

O `trigger_rule='all_success'` reforça que a task de relatório só executa se todas as tasks anteriores tiveram sucesso.

O time recebe o alerta de falha do Spark via `email_on_failure=True` e sabe que o relatório não vai sair, podendo investigar e reprocessar manualmente se necessário.

Questão 11. Kafka, em poucas linhas

O que é uma partição em um tópico Kafka e qual a relação com paralelismo no consumidor?

Um tópico Kafka é dividido em partições. Cada partição pode ser vista como uma fila ordenada e independente. Mensagens dentro de uma partição têm ordem. Entre partições diferentes, não há garantia de ordem.

A relação com paralelismo: cada partição só pode ser consumida por um consumidor dentro do mesmo consumer group ao mesmo tempo.

Tópico com 6 partições + 6 consumidores = 6 consumidores processando em paralelo
Tópico com 6 partições + 3 consumidores = cada consumidor processa 2 partições
Tópico com 3 partições + 6 consumidores = 3 consumidores ativos, 3 ociosos
número de partições = grau máximo de paralelismo.

Diferencie at-least-once e exactly-once delivery. Qual é mais comum na prática e por quê?

At-least-once: A mensagem é entregue uma ou mais vezes. O consumidor lê a mensagem, processa, e depois faz commit do offset. Se o consumidor cair após processar mas antes de fazer o commit, a mensagem será reentregue na próxima vez. Resultado: pode haver duplicatas, mas nenhuma mensagem é perdida.

Exactly-once: Cada mensagem é processada exatamente uma vez sem duplicatas e sem perdas. Requer transações coordenadas entre produtor, broker e consumidor. Muito mais complexo de implementar e com overhead de performance significativo.

O que é um consumer group e o que acontece se você adicionar mais consumidores do que partições?

Um consumer group é um conjunto de consumidores que trabalham juntos para consumir um tópico. O Kafka distribui as partições entre os consumidores do grupo e cada partição vai para um consumidor.

Consumidores extras ficam ociosos, não recebem nenhuma partição. O Kafka não divide uma partição entre dois consumidores do mesmo group.

Partição 0 → Consumidor A
Partição 1 → Consumidor B
Partição 2 → Consumidor C
Partição 3 → Consumidor D
Consumidor E → ocioso
Consumidor F → ocioso

Adicionar o 5º e 6º consumidor não aumenta o throughput e aumenta o custo. Consumidores extras são úteis quando servem de standby. Se o Consumidor A cair, o Kafka faz rebalanceamento e o Consumidor E assume a Partição 0. Alta disponibilidade sem intervenção manual.

Questão 12. Monitoramento

Que métricas você consideraria essenciais para acompanhar um pipeline de dados em produção? Cite ao menos quatro (técnicas e de negócio), explique o que cada uma diz sobre a saúde do sistema e descreva o limiar que dispararia um alerta para cada uma.

Métricas técnicas do job:

Duração do job: indica quanto tempo o pipeline levou para rodar, pode indicar degradação de performance, disco lento ou volume inesperadamente alto. Coletar no `run_daily.sh` e registrar e calcular em [/var/log/etl/run_daily.log](#)

Percentual de linhas ignoradas: indica a proporção de linhas malformadas no dia. Limiar de alerta: 5%, indica mudança no formato do log upstream, pode ser um deploy com bug na aplicação que gera os logs. Coletar em: `process_logs.py`

Exit code do job: mostra se o Job terminou com sucesso ou falha. Qualquer valor diferente de zero dispara alerta imediato. coletar em: `systemd journal — journalctl -u etl.service`

Volume de linhas processadas: mostra quantas linhas foram processadas no dia. Limiar de alerta: queda de mais de 30% em relação à média dos últimos 7 dias. Coletar em: `process_logs.py`

Taxa de erro por app: percentual de requests com status ≥ 500 . Limiar de alerta: qualquer app acima de 10% indica problema na aplicação.

Latência p99 por app: mostra os 1% de requests mais lentos. Limiar de alerta: acima de 5000ms indica alta degradação de performance

Usuários únicos por dia: mostra o volume de usuários ativos por dia. Limiar de alerta: queda de mais de 40% vs média pode indicar problema de coleta ou queda de uso da solução.

Total de requests por app: volume de tráfego. Limiar de alerta: zero requests em qualquer app. Pode estar fora do ar ou sem logs chegando.
